Image Deblur and Denoise

Comparing TpV, ResU-Net, GAN, PD-Net TV

Gruppo S : Lorenzo Davalli, Luca Di Mauro, Emiliano Chiarini

Link al repository

Project Description

Task: Deblur and Denoise

- The task must be formulated as an inverse problem where a degraded observation is mapped back to a high-quality reconstruction.
- Selected methods
  - **Variational method:** regularization with Total p-Variation with p in (0.1, 0.5)
  - **End-to-end method:** ResU-Net
  - **Generative method:** GAN with DGP
  - Hybrid Methods: PD-Net with TV
- Metrics: PSNR, SSIM, comparison plot

Deblurring and Denoising

Inverse Problem

In computational imaging, image acquisition is modeled as a degradation process:

$$y = Kx + n$$

y: Degraded observation

K: Blur operator

x: Latent sharp image (unknown)

n: Additive noise (e.g., Gaussian)

The goal is to estimate x from y, while addressing the ill-posed nature of the problem.

Main Challenges

- **Information Loss: Blur acts as a low-pass filter that removes high frequencies.**
- **Noise Amplification: The direct inversion of K tends to drastically amplify noise n.**
- **Handling ill-posedness: a priori techniques are needed to guide the reconstruction.**

Variational methods

Variational methods reconstruct the image by solving an optimization problem. They combine two terms: a **data-fidelity term**, which enforces consistency with the measured sinogram, and a **regularization term**, which encodes prior assumptions on the image.

$$\hat{x} = \arg\min_x \frac{1}{2}\|Kx - y^\delta\|_2^2 + \lambda R(x)$$

Different choices of R(x):

$$\begin{aligned}
&\text{Tikhonov} && R(x) = \|x\|_2^2 \\
&\text{Total Variation} && R(x) = \sum_i \|\nabla x_i\|_2 \\
&\text{Total } p\text{-Variation} && R_p(x) = \sum_i \|\nabla x_i\|_2^p, \quad 0.1 < p < 0.5
\end{aligned}$$

ResU-Net

End-to-end method: a neural network f_Θ is trained once on many examples to directly learn the inverse map from the degraded measurement to the clean reconstruction.

Given the degradation model $y^\delta = Kx + e$, the network is trained by minimizing the MSE between reconstruction and clean target over the training set:

$$\min_\Theta \frac{1}{N} \sum_{i=1}^{N} \| f_\Theta(y_i^\delta) - x_i \|_2^2$$

ResU-Net

Unlike variational methods, K is not used explicit: the forward model is learned implicitly from data. Residual learning: instead of reconstructing $\boldsymbol{x}$ from scratch, the network learns a correction added back to the input:

$$f_\Theta(y^\delta) = y^\delta + r_\Theta(y^\delta)$$

The residual is easier to learn and generalizes better than the full image, since artifact patterns are task-related rather than dataset-specific.

U-Net backbone: multi-scale encoder-decoder with skip connections: large receptive field for global artifacts, fine detail preserved for sharp edges.

GAN

We trained a **GAN** to learn the distribution of the clean images.

Given a latent vector **z** the generator produces an image

$$\hat{x} = G(z)$$

while the critic evaluates how realistic is the generated image is

$$D(\hat{x}, x^{GT})$$

WGAN-GP as Generative Prior

We specifically trained a **Conditional Wasserstein GAN with Gradient Penalty** model:

- Wasserstein Distance: Provides a smoother gradient descent and more stable training by changing the GAN objective to the Earth-Mover distance
- Gradient Penalty: Improves the convergence and avoid unstable critic behaviour

$$L_{GP} = \lambda(\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1)^2$$

- Diff Augment: Data augmentation applied to both real and generated images
- Exponential Moving Average: Maintain a moving average of the weights of the Generator

PD-Net with TV

It’s an unrolled optimization network that merges deep learning with classical variational methods. It replaces the data fidelity and primal update steps with trainable CNNs to adapt to complex noise and blur, while strictly enforcing Total Variation (TV) as a regularizer by computing its exact analytical proximal step in the dual gradient space.

- Algorithm Unrolling: The architecture maps the iterative Primal-Dual Hybrid Gradient (Chambolle-Pock) algorithm into a deep neural network, unrolling a fixed number of optimization steps.
- Hybrid Dual Space: The dual variables are split into two distinct branches to treat data fidelity and regularization separately.
- Learned Data Fidelity: The dual proximal operator for data fidelity (MSE) is replaced by a Convolutional Neural Network (CNN).
- Analytical TV Regularization: The Total Variation (TV) regularizer is kept analytical. Its dual proximal step is computed mathematically via exact geometric projection onto the L infinity ball.
- Learned Primal Update: A primal CNN recombines the adjoint operators of both dual variables to iteratively reconstruct the final clean image.

Dataset

Dataset to be used: ImageNet 1k 256x256

It’s a RGB dataset containing 256x256 images, divided in 3 sections:

- Train (1281167 images)
- Test (50000 images)
- Validation (100000 images)

Since the dataset is too big, the dataset has been reduced in size of (1300 images for each class for 4 classes as required by the professor) while keeping the proportions of 80% for train, 10% for test and 10% for validation

Preprocessing

The dataset has been preprocced to add the blur and noise to the images.

Gaussian Blur:

sigma = 2, kernel size = 9

noise level = 0.005, 0.01, 0.05, 0.1

In addition all images have been normalized to [0,1]

Tvp Implementation

The deblurring and denoising problem is solved by utilizing the Total p Variation.

The Total p Variation has been implemented with the Chambolle-Pock algorithm.

Chambolle-Pock alone is not capable of implementing TpV, because TpV is non-convex.

To solve this problem the Iterative Reweighting has been integrated to the classic Chambolle-Pock algorithm. Iterative Reweighting transforms the difficult Lp problem into a sequence of weighted L1 problems, which are perfectly convex and can be solved using the Chambolle-Pock method

Iterative Weighting

Iterative Reweighting replaces the p norm of TpV with the weighted norm L1, where the weights are:

$$w_i^{(k)} = \frac{1}{(|\nabla x_i^{(k-1)}| + \epsilon)^{1-p}}$$

The effect of the weight w is the same as the TpV, if, in the previous step, the algorithm detected a sharp edge, the denominator increases and the weight w becomes small. In the next iteration, the algorithm will apply less regularization precisely to that edge.

Conversely, in a flat area with noise, the weight w becomes large, forcing the algorithm to smooth out the area aggressively.

TvP parameters

The key aspect in order to reconstruct good images with TvP is to fine tune the main parameters of the system:

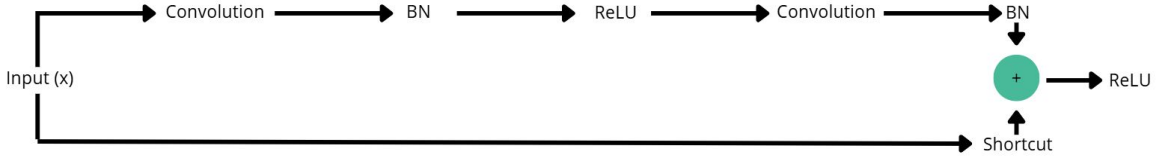
	0,005	0,01	0,05	0,1
Lambda	7e-5 ± 3e-5	5e-4 ± 1e-5	7e-3 ± 3e-3	4e-2 ± 1e-2
p	0,5	0,5	0,5	0,5
# of iterations	~3000	~500	~100	~50

NOTE: These values are indicative and general, to achieve the best performance on an image, you'll have to fine tune these parameters on that particular image.

In addition the starting point of the algorithm is the degraded image y.

ResU-Net Implementation

It is a symmetric encoder-decoder (UNet-style) where each convolutional block is replaced by a **Residual Block**.



- **Encoder**: 4 downsampling levels $(\mathrm{MaxPool}\ \ 2 \times 2)$ that double the channels at each step: $32 \rightarrow 64 \rightarrow 128 \rightarrow 256 \rightarrow 512$ (bottleneck).
- **Decoder**: 4 upsampling levels (*ConvTranspose2d*) with skip connections via concatenation (classic UNet style), with channels symmetric to the encoder.
- **Output**: Final 1×1 conv + Sigmoid to constrain the output within $[0,1]$ (consistent with normalized RGB images).

ResU-Net Implementation

Training Loop:

BATCH_SIZE	8
EPOCHS_NUMBER	30
Optimizer	AdamW
Learning Rate	1e-3
Loss	MSE
Early Stopping patience	7
Scheduler patience	3

WGAN-GP Implementation

The dataset used here is resized to 64x64 pixel, for time and machine constraints.

Setup: 350 epochs, 64 img per batch, 1 Critic step / 1 Generator step

Two Time-Scale Update of $lr_D = 3 \times 10^{-4}, lr_G = 1 \times 10^{-4}$ with a **cosine decay** starting at 80% of the epochs, with floor=0.5

For the optimizer, we have chosen **Adam** with $\beta_1 = 0.0, \beta_2 = 0.9$

WGAN-GP Implementation

The dataset used here is resized to 64x64 pixel, for time and machine constraints.

Generator: $G(z) : z \sim \mathcal{N}(0, I) \in \mathbb{R}^{128} \xrightarrow{\text{FC}} 4 \times 4 \times 512 \xrightarrow{\text{4 x ResBlockUp}} \xrightarrow{\text{Conv 3x3 + tanh}} \text{IMG } 64 \times 64 \times 64$

ResBlockUp: Nearest Upsampling 2x, BatchNorm, Residual Connection

Critic: $D(x) : 64 \times 64 \times 3 (+ \text{ DiffAugment}) \xrightarrow{\text{Conv 3x3}} \xrightarrow{\text{4x ResBlockDown}} \xrightarrow{\text{FC}} \text{Score} \in \mathbb{R}$

ResBlockDown: AvgPool 2x, LeakyRelu, No BatchNorm

With a loss of: $\mathcal{L}_D = \mathbb{E}[D(G(z))] - \mathbb{E}[D(x_{GT})] + 10 \cdot \mathbb{E}[(\nabla_{\hat{x}} D(\hat{x})||_2 - 1)^2]$

$\mathcal{L}_G = -\mathbb{E}[D(G(z))]$

PD-Net Implementation

As an unrolling algorithm, PD-Net's implementation follows three main computational steps for each unrolled iteration:

1. Data Fidelity Dual Update (Learned via CNN)

The first branch handles the data fidelity term. Instead of computing an analytical proximal step, this update is learned.

The network computes the forward projection of the current image(Ax), concatenates it with the current dual state (p_data) and the original degraded measurements (y), and passes them through a CNN block (DualNet). The CNN predicts a residual update to refine the p_data variable.

PD-Net Implementation

2. Total Variation Dual Update (Analytical Proximal Step)

The second branch enforces piecewise smoothness using Total Variation (TV). This step is kept analytical. The implementation:

- Computes the spatial gradients (grad_x) of the current image using finite differences.
- Performs a gradient ascent step in the dual space scaled by a learnable parameter (sigma_tv).
- Applies the exact mathematical proximal operator of the L1 norm, which corresponds to a geometric projection (clipping) onto the L infinity ball defined by a learnable regularization weight (lambda_tv).

PD-Net Implementation

3. Primal Update (Learned via CNN)

The final step reconstructs the actual image (the primal variable x). First, the dual variables are mapped back to the image space using their respective adjoint operators:

- The transposed blur operator (A^T) is applied to p_data.
- The adjoint of the gradient is applied to p_tv to project the TV dual variable in the primal space.

These back-projected tensors are concatenated with the current image estimate x and fed into a second CNN block (PrimalNet). The network learns to optimally fuse the data-fidelity corrections and the TV-regularized edges, outputting the updated, cleaner image for the next iteration.

RESULTS

Image Results Class ‘Fish’: Total p Variation

Image Results Class ‘Fish’ : ResU-Net

Ground Truth

Image Results Class ‘Fish’ : WGAN-GP

Image Results Class ‘Fish’ : PD-Net

Image Results Class ‘Dog’: Total p Variation

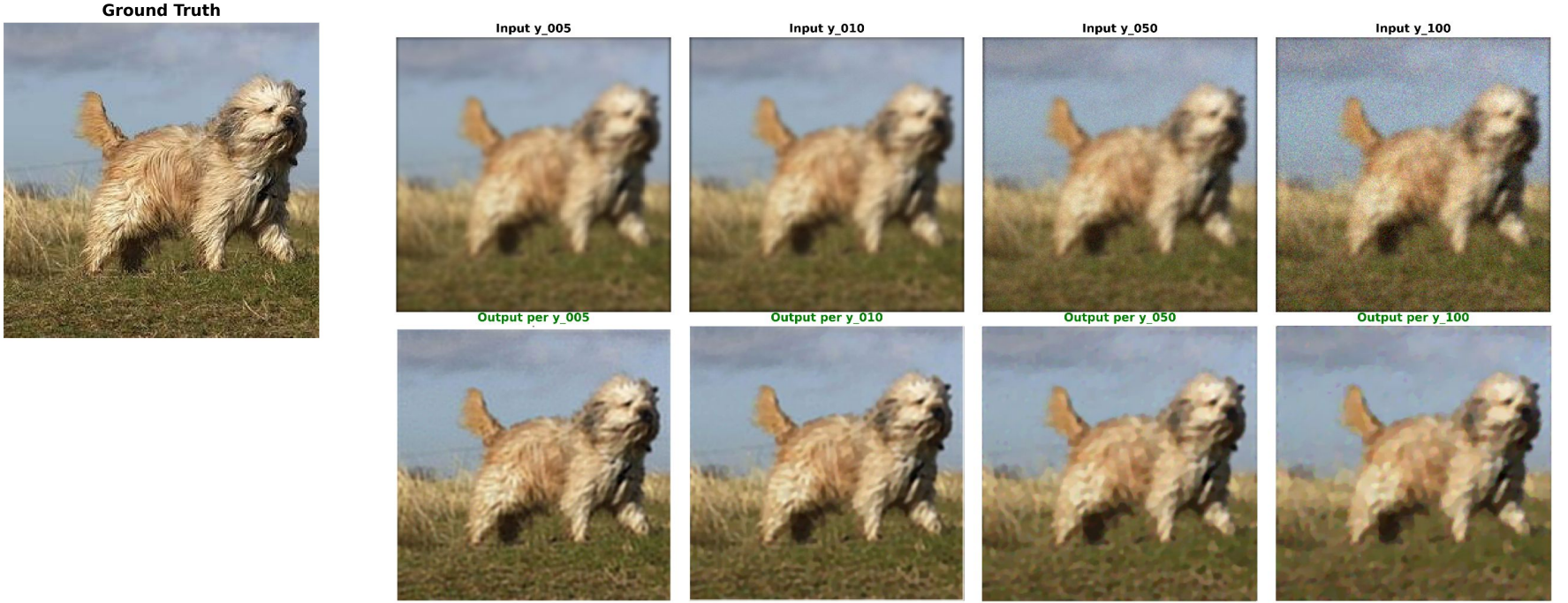


Image Results Class ‘Dog’: ResU-Net

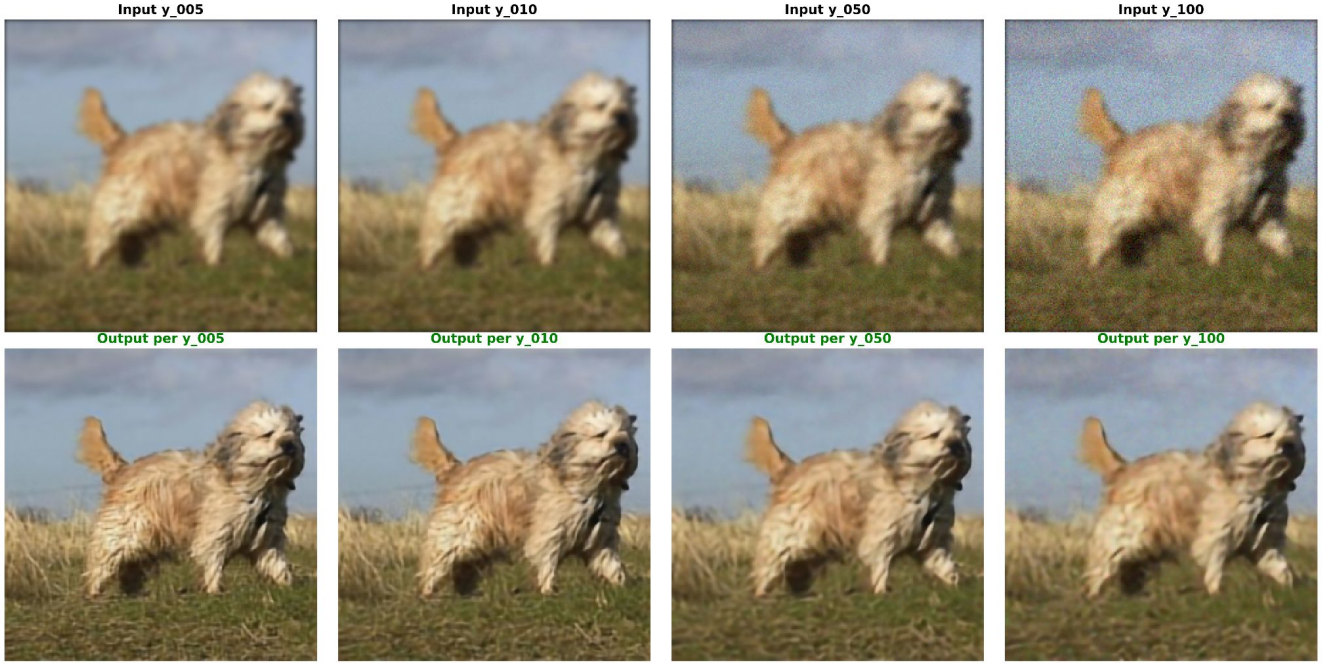


Image Results Class ‘Dog’: WGAN-GP

Image Results Class ‘Dog’: PD-Net

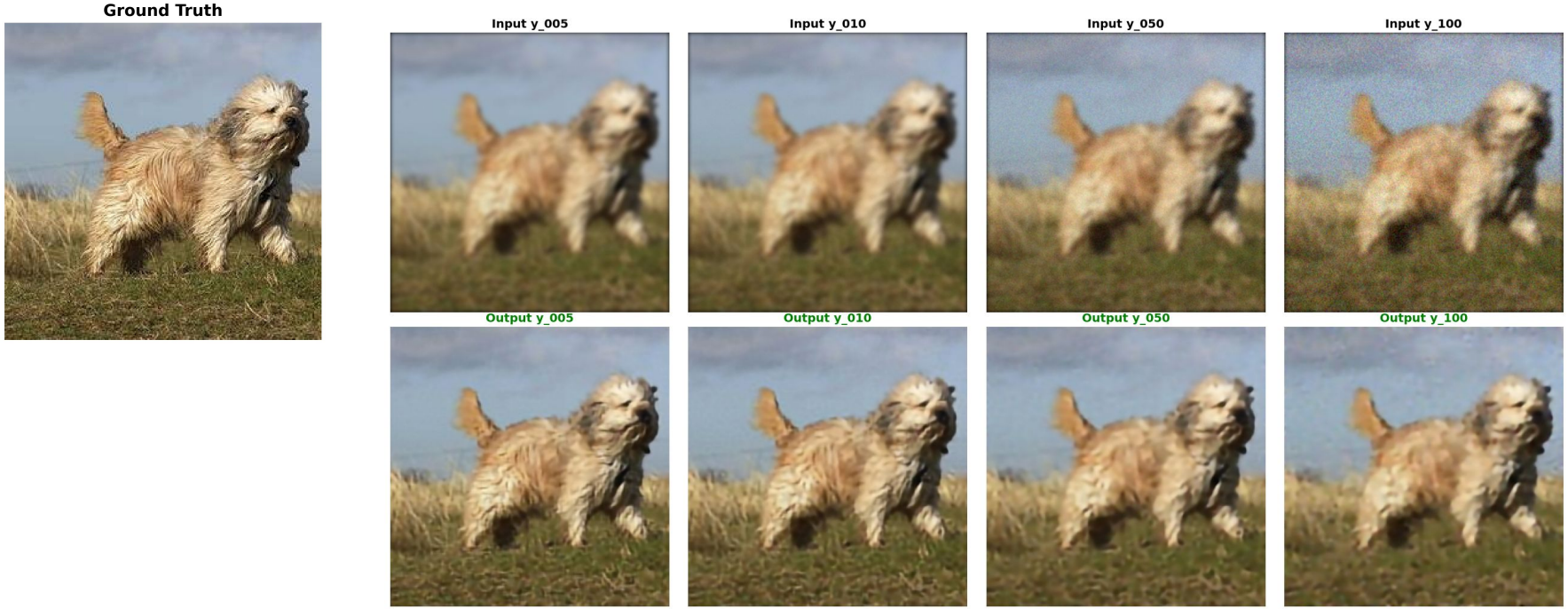


Image Results Class ‘Bird’: Total p Variation

Image Results Class ‘Bird’: ResU-Net

Ground Truth

Image Results Class ‘Bird’: WGAN-GP

Image Results Class ‘Bird’: PD-Net

Image Results Class ‘Swan’: Total p Variation

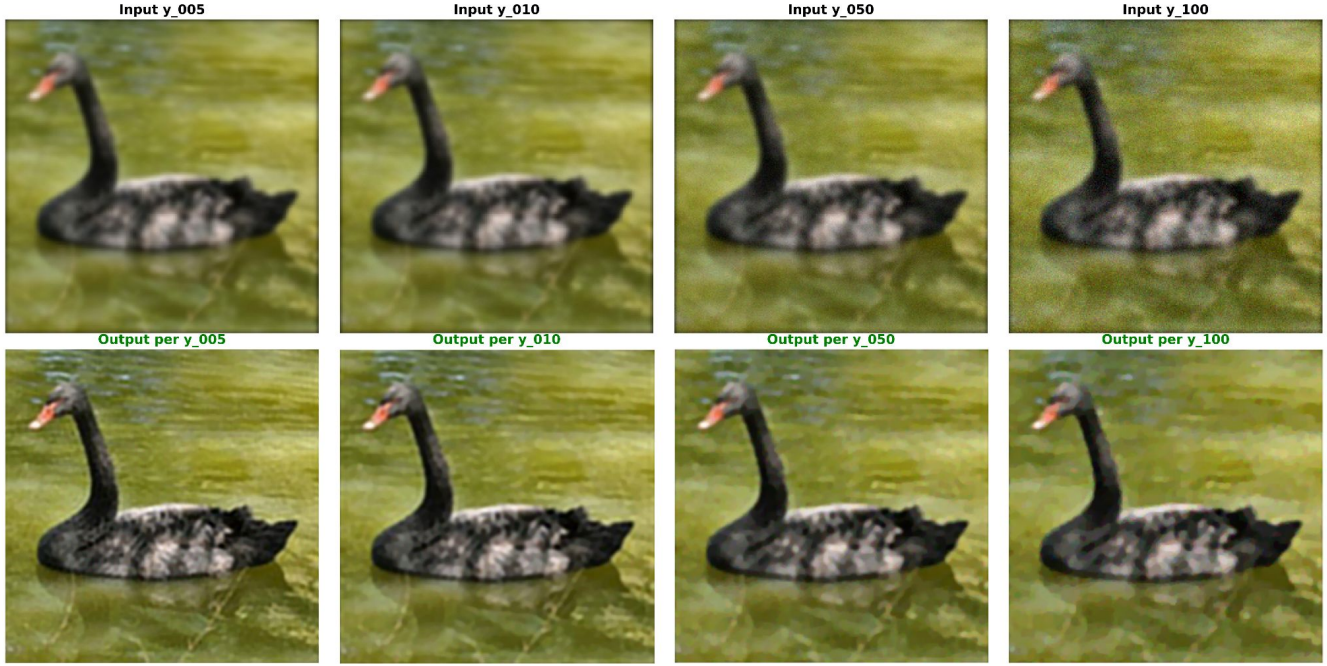


Image Results Class ‘Swan’: ResU-Net

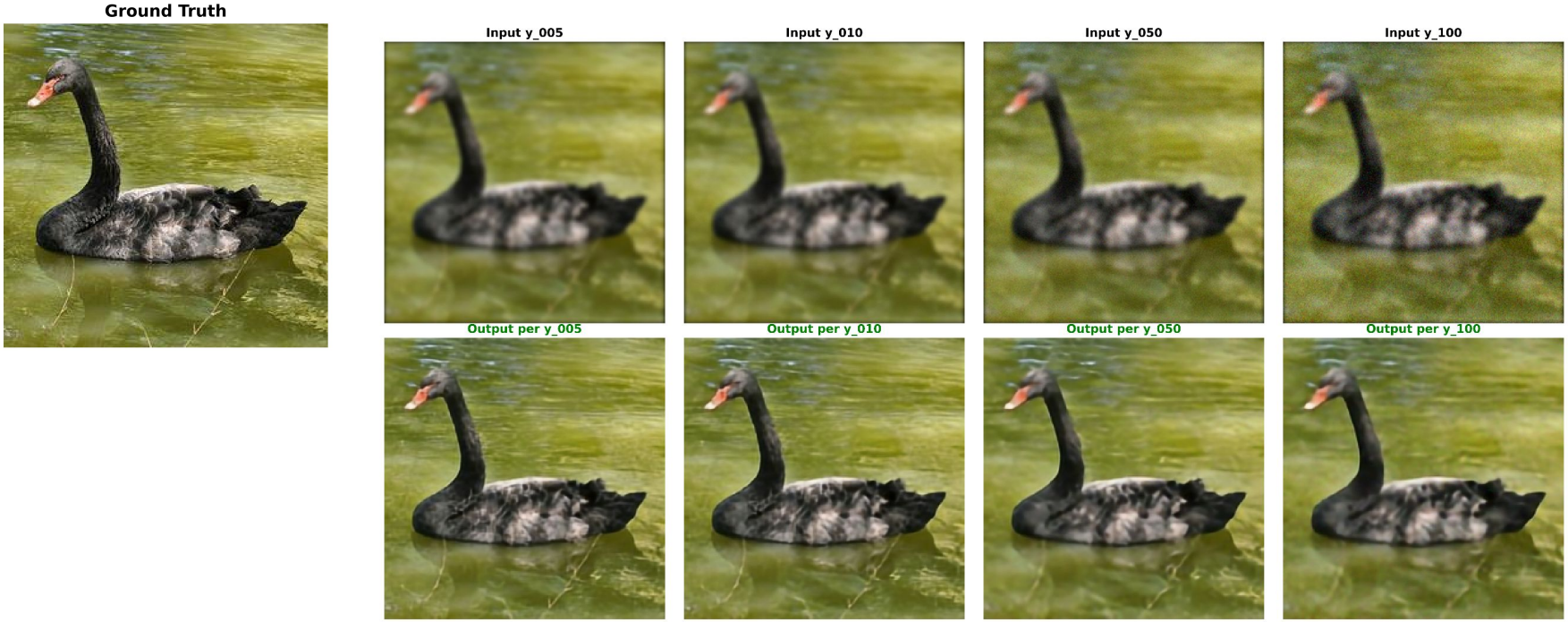


Image Results Class ‘Swan’: WGAN-GP

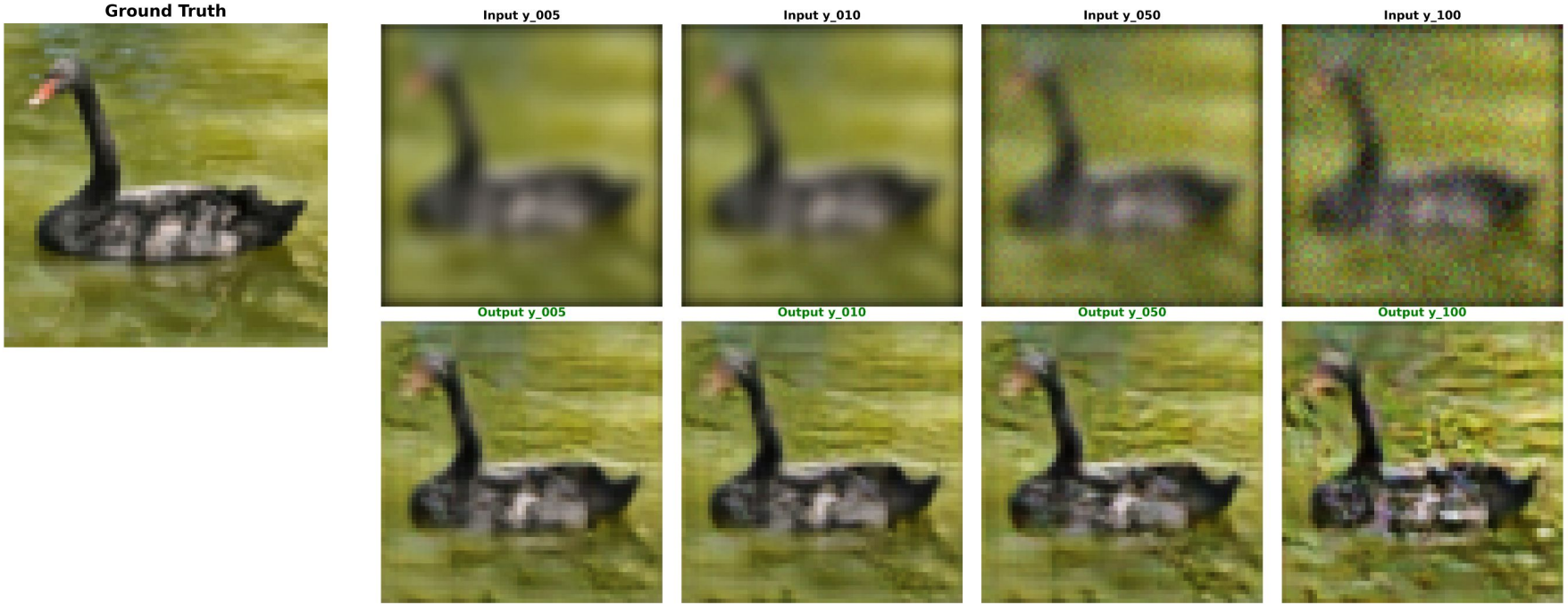
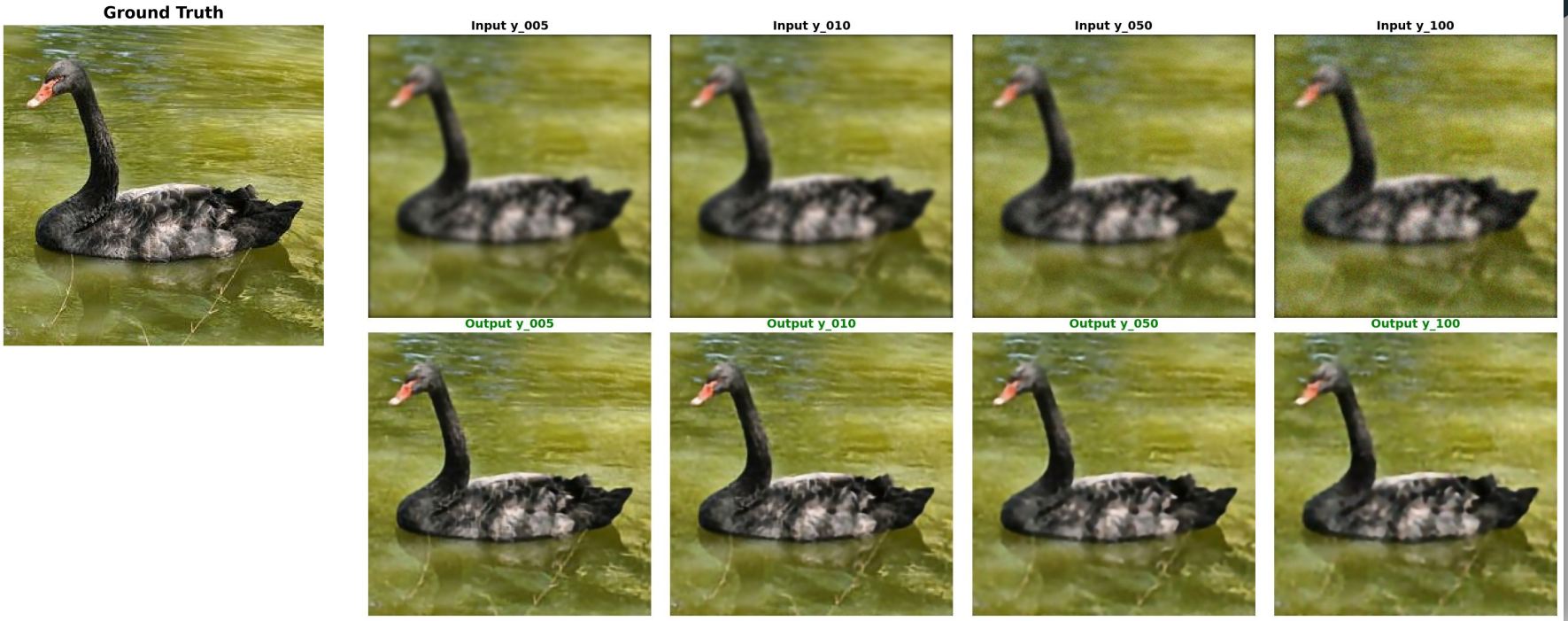


Image Results Class ‘Swan’: PD-Net



Numerical Results

	0.005	0.01	0.05	0.1
TpV PSNR	26,6 ± 3,61	26 ± 3,5	25,59 ± 2,48	24,48 ± 2,81
ResUnet PSNR	28,34 ± 3,35	27,89 ± 3,34	26,61 ± 3,2	25,86 ± 3,01
GAN PSNR	26,48 ± 2,68	26,43 ± 2,72	25,29 ± 2,49	22,42 ± 2,34
PD-Net PSNR	28.81 ± 3.72	28.18 ± 3.66	26.72 ± 3.4	25.89 ± 3.13
TpV SSIM	0,7982 ± 0,083	0,7848 ± 0,101	0,7005 ± 0,086	0,6527 ± 0,114
ResUnet SSIM	0,8533 ± 0,079	0,8361 ± 0,086	0,7757 ± 0,103	0,7309 ± 0,108
GAN SSIM	0,8001 ± 0.0602	0,7969 ± 0,0611	0,7491 ± 0,0568	0,6260 ± 0,0619
PD-Net SSIM	0.8565 ± 0.0781	0.8356 ± 0.0869	0.7724 ± 0.1042	0.7269 ± 0.1072

Conclusions Variational TpV

Classic model-based iterative methods utilizing Total p Variation are nowadays outclassed by the new DL-based methods in terms of performance and convenience of use.

The only advantage that classic methods have is when you want to clean a particular image, in that case classic methods are way faster and can be tuned to achieve great results.

Conclusions ResU-Net

ResU-Net, as a deep-learning end-to-end method, outclasses both classic variational methods (TpV) and generative approaches (GAN) in terms of raw reconstruction quality, consistently achieving the best PSNR and SSIM across generative and model based techniques.

Its main advantage is convenience: once trained, it reconstructs any image in a single forward pass, with no per-image tuning required. The trade-off is upfront cost, it needs a representative training dataset and a training phase before it can be used, unlike TpV which can be applied directly to a single image with no prior training at all.

Conclusions GAN

The GAN used as a Deep Generative Prior (DGP) proves competitive with the classical variational TpV method at low to moderate noise levels. For $\sigma = 0.005–0.05$, it achieves a PSNR of approximately 25–26.5 dB, comparable to TpV (26.6 → 25.6 dB), and in some cases attains a higher SSIM (0.749 vs. 0.701 at $\sigma = 0.05$). However, it remains about 2 dB below the supervised methods ResUNet and PD-Net, which are the best-performing approaches (~28 dB at low noise levels), benefiting from access to degraded images during training.

The main limitation of the GAN appears at high noise levels ($\sigma = 0.1$), where performance drops to 22.42 dB and 0.626 SSIM because the fine-tuning process struggles to distinguish image structure from noise. In conclusion, with limited data and in an unsupervised setting, the generative prior achieves a reconstruction quality comparable to that of a variational solver, but it does not match the performance of supervised neural networks and is less robust when noise dominates the measurements.

Conclusion PD-Net

Hybrid methods in general show results above the other methods.

But the time needed to train the model and to run it, was considerably longer than any of the other methods due to the need to pre-train the CNNs and the time that the actual unrolling algorithm takes to execute.

So hybrid methods are really good in terms of performance but requires more time and consideration than other methods.

The END